DevCraft: A Guided MERN Stack Learning Platform & Developer Community

Document ID:	DEVCRAFT-SRS-IN2901-V1.0	**Academic Module:**	IN2901 — Software Development
Development Team:	Team 22 (Team DoD)	**Degree Programme:**	B.Sc. (Hons) in Information Technology
Academic Supervisor:	Ms. M. N. Chandimali	**Industry Mentor:**	Mr. H. K. Gaminda Ishara (Creative Software)
Baseline Version:	1.0 (Formal IEEE 830 Release)	**Release Date:**	October 2026

DevCraft: A Guided MERN Stack Learning Platform & Developer Community

Document Identifier: DEVCRAFT-SRS-IN2901-V1.0
Standard: IEEE Std 830-1998 / ISO/IEC/IEEE 29148:2018
Date: October 2026
Module: IN2901 — Software Development Project
Degree Programme: B.Sc. (Hons) in Information Technology
Faculty / University: Faculty of Information Technology, University of Moratuwa
Project Team: Team 22 · Team DoD
Academic Supervisor: Ms. M. N. Chandimali
Industry Mentor: Mr. H. K. Gaminda Ishara (Creative Software)

Subsystem Responsibilities & Authors

Index No	Member Name	Subsystem & Feature Responsibility
244068K	Gunasekara M.I.T.	Feature 01: The Guided Coding Workspace & Anti-Boredom Engine
244230C	Wijesiri K.S.K.	Feature 02: Theory Dojo (Curriculum, Video Checkpoints & Code Puzzles)
244118P	Madhushan M.A.	Feature 03: The Debugging Simulator & Competitive Leaderboard
244096T	Kalhari L.T.	Feature 04: Dual-Purpose Developer Community (Blogs & Real-Time Chat)
244047V	Dinisuru P.W.L.	Feature 05: Hybrid Mentorship Ecosystem & Discount Rewards

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) establishes the formal functional, behavioral, performance, and environmental requirements for **DevCraft**. DevCraft is a unified, gamified web learning ecosystem and developer community engineered specifically to address the steep learning curve, "tutorial hell," and high cognitive friction experienced by university students and junior developers learning the **MERN** (MongoDB, Express.js, React, Node.js) technology stack.

This document serves as the formal contractual and engineering baseline for the development team, academic supervisors, industry mentors, and quality assurance assessors throughout the IN2901 Software Development lifecycle.

1.2 Document Conventions

- **RFC 2119 Standard:** Requirements use normative keywords:
  - **SHALL / MUST:** Mandatory requirement.
  - **SHOULD / RECOMMENDED:** High priority, expected unless technical barriers exist.
  - **MAY / OPTIONAL:** Permissible feature or enhancement.
- **Requirement Tagging Convention:**
  - `[DEV-FR-F01-XXX]`: Functional Requirement for Feature 01 (Workspace)
  - `[DEV-FR-F02-XXX]`: Functional Requirement for Feature 02 (Theory Dojo)
  - `[DEV-FR-F03-XXX]`: Functional Requirement for Feature 03 (Debugging Simulator)
  - `[DEV-FR-F04-XXX]`: Functional Requirement for Feature 04 (Community & Chat)
  - `[DEV-FR-F05-XXX]`: Functional Requirement for Feature 05 (Mentorship & Rewards)
  - `[DEV-NFR-XXX]`: Non-Functional Requirement

1.3 Intended Audience

- **Developers & Subsystem Leads:** For technical implementation and component contracts.
- **Academic Supervisor & External Examiners:** To assess compliance with IN2901 project requirements.
- **Industry Mentor (Creative Software):** For architectural validation against software industry standards.
- **QA Engineers & Testers:** For acceptance test case formulation and verification.

1.4 Product Scope

DevCraft provides an integrated, five-pillar pedagogical environment:

1. **Interactive Guided Coding Workspaces:** In-browser multi-panel IDE (Monaco Editor, live sandboxed execution, real-time Abstract Syntax Tree validation, Jest behavioral tests, and dynamic fatigue-relief meme interventions).
2. **Theory Dojo:** Structured level-based video lectures with timestamped in-video quiz checkpoints, fill-in-the-blank code puzzles, and streak retention tracking.
3. **Debugging Simulator:** Gamified time-attack scenario challenges providing intentional code mutations, real-time error log simulation, three-heart life penalties, and XP rewards.
4. **Dual-Purpose Community:** Technical blogging with markdown support, categorized discussions, and low-latency WebSocket chat rooms for group debugging and peer exchange.
5. **Hybrid Mentorship & Rewards:** Consumable coin wallet powering instant AI code assistance, 1-on-1 human mentor scheduling with PayHere payment integration, and performance-driven discount voucher unlocking.

1.5 Definitions, Acronyms, and Abbreviations

- **AST (Abstract Syntax Tree):** Tree representation of the abstract syntactic structure of JavaScript source code generated via Babel/ESTree parsers.
- **CPM:** Critical Path Method for project task tracking.
- **MERN:** MongoDB, Express.js, React.js, Node.js full-stack software stack.
- **Monaco Editor:** The browser-based code editor that powers Microsoft Visual Studio Code.
- **JWT (JSON Web Token):** Compact URL-safe means of representing claims between two parties.
- **PayHere:** Central Bank of Sri Lanka approved online payment aggregator.
- **XP (Experience Points):** Quantifiable unit measuring learner platform progress and achievement.

---

2. Overall Description

2.1 Product Perspective & Multi-Tier Architecture

DevCraft is designed as a cloud-native single-page web application (SPA) backed by micro-services, real-time socket clusters, and sandboxed code execution workers.

Architectural Tiers Breakdown

Architectural Tier	Components & Technologies	Primary Responsibilities & Data Flow
Tier 1: Client Presentation Layer	React.js, Tailwind CSS, Monaco Editor API, Framer Motion, HTML5 Canvas	Renders the interactive UI, manages local workspace state via Zustand, captures student code inputs, and runs client-side AST syntax checks.
Tier 2: API Gateway & Application Router	Node.js, Express.js, JWT Authentication Middleware, Express Validator	Serves as the central API gateway; handles user authentication, routes RESTful requests, enforces rate-limiting, and controls business logic.
Tier 3: Real-Time Services & Worker Pool	Socket.IO Server, Isolated Code Sandbox (`isolated-vm` / Child Worker Pool)	Manages real-time bidirectional messaging for community rooms and executes student code behavioral tests in isolated, resource-capped sandboxes.
Tier 4: Persistence & External Integrations	MongoDB Atlas, Redis 7.x, PayHere Gateway, Google Gemini / OpenAI API	Stores relational documents (users, progress, lessons), caches real-time leaderboards in Redis sorted sets, verifies online payments, and powers AI mentor responses.

(For full structural design class diagrams, refer to Master Class Diagram and Master ER Diagram).

2.2 System Functions Summary

Subsystem	Primary Core Functions
01. Guided Workspace	Tri-panel IDE, live DOM sandbox, background AST validation, Jest test harness, prerequisite checks, contextual meme popup manager.
02. Theory Dojo	Modular curriculum, video player with interactive pause checkpoints, fill-in-the-blank code engine with regex evaluation, streak manager.
03. Debugging Simulator	Timed challenge generator, 3-lives game loop, simulated stdout/stderr error logs, XP computation, dynamic leaderboard rankings.
04. Developer Community	Markdown technical blog publishing, hashtag indexing, upvoting, nested comments, WebSocket group rooms, mentor badge accreditation.
05. Mentorship & Rewards	Coin wallet management, Gemini AI code queries, mentor calendar slot booking, PayHere payment processing, XP-driven discount vouchers.

2.3 User Classes and Characteristics

User Persona	Characteristics & Educational Level	Responsibilities & System Interactions
Student Learner	University undergraduate or novice developer learning MERN stack; high susceptibility to cognitive fatigue.	Solves guided projects, watches Dojo videos, battles bugs in simulator, seeks AI/human mentor guidance, earns XP.
Verified Mentor	Senior undergraduate, software engineer, or industry professional verified via portfolio.	Sets availability calendar slots, conducts 1-on-1 coaching sessions, answers community inquiries, reviews student code.
System Administrator	Course supervisor or platform maintainer.	Curates curriculum, manages challenge test suites, moderates community reports, configures PayHere coin packages and discount policies.

2.4 Operating Environment

- **Client Tier:**
  - Modern HTML5/ES6+ web browsers: Chrome 110+, Firefox 112+, Edge 110+, Safari 16+.
  - Minimum display resolution: 1280 × 800 (Recommended: 1920 × 1080 for multi-panel IDE).
- **Server Tier:**
  - Node.js LTS (v18.x / v20.x / v24.x) running on Linux x86_64 or containerized Docker.
  - Express.js 4.x REST APIs with Socket.IO 4.x.
- **Database & Cache:**
  - MongoDB Atlas 6.0+ (Replica Set enabled).
  - Redis 7.x for rate limiting, active socket sessions, and sorted sets (ZSET) for global leaderboards.

2.5 Design and Implementation Constraints

- **Client Sandboxing Safety:** User-submitted code executed in the browser live preview MUST run within an isolated `<iframe>` with strict `sandbox="allow-scripts"` attributes and blocked access to the parent window `localStorage` and authentication tokens.
- **Backend Code Execution Safety:** Server-side test executions MUST run within isolated worker threads (`isolated-vm` or restricted child processes) with strict timeouts (≤ 3000 ms) and memory caps (≤ 128 MB) to eliminate runaway loops and security vulnerabilities.
- **Regulatory & Currency Compliance:** Payment processing MUST interface with the PayHere Sandbox and Live gateways in Sri Lankan Rupees (LKR) adhering to Merchant Security Guidelines.

---

3. External Interface Requirements

3.1 User Interfaces

3.1.1 UI-01: Tri-Panel Guided Coding Workspace

- **Panel 1 (Left - Instructions):** Step title, markdown pedagogical guidance, collapsible code examples, progressive hints (3 tiers).
- **Panel 2 (Center - Monaco Editor):** Full-featured code editor with syntax highlighting, line numbers, automatic bracket matching, and in-editor syntax diagnostics.
- **Panel 3 (Right - Live Preview & Diagnostics):** Real-time rendered React/HTML component, captured console output, and visual AST validation feedback cards (Green/Red checkmarks).
- **Overlay: Contextual Meme Modal:** Non-intrusive modal triggered upon consecutive failed attempts or prolonged inactivity, featuring humorous programming memes to relieve compiler anxiety.

3.1.2 UI-02: Theory Dojo Interface

- Video playback container with custom scrubber that locks progress until in-video interactive questions are answered correctly.
- Code puzzle panel featuring inline editable input blanks within formatted syntax blocks.

3.1.3 UI-03: Debugging Simulator Arena

- Top HUD displaying: Remaining Time (digital countdown with red pulse when under 60 seconds), 3 Heart Life Containers, and Total Bounty XP.
- Simulated Terminal window rendering formatted stack traces, Jest failure outputs, and compiler error snippets.

3.1.4 UI-04: Community Hub & Chat

- Dual-tab navigation between "Technical Articles" (card feed with cover images, read times, and author badges) and "Live Discussion Channels" (categorized topic rooms with active user lists).

3.1.5 UI-05: Mentorship Booking & Wallet Portal

- Mentor profile cards with star ratings, technical badges, hourly rates in coins, and interactive weekly time-slot picker.
- Persistent Coin Wallet widget in the top navigation bar showing current coin balance and 1-click "Top Up" modal.

3.2 Software Interfaces

- **Monaco Editor API:** Web-worker driven language services for JavaScript and TypeScript syntax parsing.
- **Babel Parser (`@babel/parser`):** AST generator analyzing structural semantics (e.g., verifying that a student declared a `useState` hook with proper destructuring).
- **PayHere Payment API:** Merchant checkout redirection, MD5 signature generation, and server-to-server IPN (Instant Payment Notification) verification.

- **Google Gemini 1.5 / OpenAI API:** Low-latency conversational endpoint configured with educational system prompts to guide students without handing them complete solutions.

---

4. System Features & Functional Requirements

Subsystem Mapping Overview

Subsystem Pillar	Module Owner	Primary Responsibilities	Core Classes & Entities
Feature 01: Guided Workspace	Gunasekara M.I.T. (244068K)	Monaco IDE, AST Syntactic Check, Jest Runner, Anti-Boredom Memes	`IDEWorkspace`, `ASTValidationService`, `ProjectStepModel`, `MemeModel`
Feature 02: Theory Dojo	Wijesiri K.S.K. (244230C)	Video Checkpoints, Code Puzzles, Resumable Bookmarks, Daily Streaks	`Lesson`, `VideoCheckpoint`, `Question`, `AcceptedAnswer`, `DailyStreak`
Feature 03: Debugging Arena	Madhushan M.A. (244118P)	Time-Attack Challenges, 3-Heart Life Loop, Error Logs, Leaderboard	`DebugSession`, `DebugChallenge`, `XPEvent`, `LeaderboardService`
Feature 04: Community & Chat	Kalhari L.T. (244096T)	Technical Blogs, Markdown Editor, Socket Discussion Rooms, Moderation	`Blog`, `Comment`, `DiscussionChat`, `ChatMessage`, `VerifiedBadge`
Feature 05: Mentorship & Coins	Dinisuru P.W.L. (244047V)	Coin Wallet, AI Mentor Querying, PayHere Gateway, Discount Rewards	`CoinWallet`, `AIConversation`, `AvailabilitySlot`, `DiscountReward`

4.1 Feature 01: The Guided Coding Workspace & Anti-Boredom Engine

(Subsystem Lead: Gunasekara M.I.T. — 244068K)

4.1.1 Description

Provides an intelligent in-browser developer environment that guides students through multi-step MERN projects. It pairs syntactic validation (AST) with behavioral tests and intervenes with psychological fatigue-relief memes when frustration is detected.

4.1.2 Functional Requirements

- **[DEV-FR-F01-001] Step-by-Step Curriculum Delivery:** The system SHALL render project steps sequentially. Step N+1 MUST remain locked until Step N passes all required syntactic and behavioral assertions.
- **[DEV-FR-F01-002] Multi-Panel Code Editing:** The system SHALL provide an integrated Monaco Editor instance supporting JavaScript, JSX, CSS, and JSON syntax.
- **[DEV-FR-F01-003] Dual-Stage Code Evaluation Engine:**
  - **Stage 1 (AST Syntactic Verification):** The system SHALL parse student code using `@babel/parser` into an AST and match nodes against expected structural rules (e.g., verifying required Express route signatures, React prop types, or Mongoose schema keys) without executing code.

    - **Stage 2 (Behavioral Unit Testing):** The system SHALL execute sandboxed Jest assertions to verify functional output.

- **[DEV-FR-F01-004] Real-Time Sandboxed Preview:** The system SHALL compile and inject student JSX/HTML/CSS into a sandboxed `<iframe>` with sub-second live re-rendering upon code change.
- **[DEV-FR-F01-005] Prerequisite Conceptual Gatekeeping:** Prior to unlocking select complex milestones, the workspace SHALL trigger a `ContextualCheckModal` requiring the user to correctly identify a core conceptual pattern.
- **[DEV-FR-F01-006] Anti-Boredom & Fatigue Detection:** The system SHALL calculate a real-time fatigue score based on:
  Fatigue Score = (0.50 × Failed Attempts) + (0.30 × Inactive Stagnation Time) + (0.20 × Rapid Deletion Events)
  When the fatigue score exceeds the threshold (≥ 0.75), the `MemePopupManager` SHALL display a humorous, contextual programming meme to break frustration loops.
- **[DEV-FR-F01-007] Workspace State Persistence:** The system SHALL autosave student code locally every 15 seconds and sync progress to the backend `LearningProgressModel` upon successful step completions.

4.2 Feature 02: Theory Dojo

(Subsystem Lead: Wijesiri K.S.K. — 244230C)

4.2.1 Description

Delivers foundational MERN stack theoretical instruction through modular interactive video lessons, automated comprehension checkpoints, fill-in-the-blank code puzzles, and streak retention gamification.

4.2.2 Functional Requirements

- **[DEV-FR-F02-001] Structured Curriculum Roadmap:** The Dojo SHALL organize lessons into progressive tracks (e.g., *React Hooks Fundamentals*, *Express Middleware Pipeline*, *MongoDB Aggregations*).
- **[DEV-FR-F02-002] Interactive Video Checkpoints:** The video player SHALL automatically pause at predefined timestamps (`VideoCheckpoint`), locking the video until the learner submits a valid response to an embedded comprehension question.
- **[DEV-FR-F02-003] Fill-in-the-Blank Code Puzzle Engine:** The system SHALL present code snippets with omitted tokens (`Blank`). It SHALL validate user-entered strings against accepted token lists and regex patterns (`AcceptedAnswer`).
- **[DEV-FR-F02-004] Resumable Learning Position:** The system SHALL persist a `LearningPosition` entity recording the active lesson, video playback timestamp (in seconds), and active tab, allowing seamless resumption across devices.
- **[DEV-FR-F02-005] Daily Learning Streak Tracker:** The system SHALL increment a learner's `DailyStreak` upon completing at least one lesson or puzzle within a 24-hour UTC window. Missing a window resets the streak unless a "Streak Freeze" is active.

4.3 Feature 03: The Debugging Simulator & Gamified Leaderboard

(Subsystem Lead: Madhushan M.A. — 244118P)

4.3.1 Description

A gamified arena designed to extinguish compiler anxiety by placing students into high-pressure, time-bounded debugging scenarios featuring realistic bugs, simulated terminal errors, and competitive global leaderboard scoring.

4.3.2 Functional Requirements

- **[DEV-FR-F03-001] Time-Attack Challenge Loop:** The simulator SHALL present faulty MERN code repositories paired with an active countdown timer (60 to 300 seconds depending on difficulty tier).
- **[DEV-FR-F03-002] Three-Heart Life Penalty System:** Each learner starts with three (3) hearts per session. Submitting code that fails challenge test cases SHALL consume one heart. Depleting all three hearts terminates the session with zero XP bounty.
- **[DEV-FR-F03-003] Realistic Error Log Simulation:** The interface SHALL inject simulated terminal and browser console outputs (e.g., `UnhandledPromiseRejection`, `Cannot read property of undefined`, `CORS header missing`) to mirror production debugging

environments.

- **[DEV-FR-F03-004] XP Bounty Calculation:** Upon fixing all test cases, the system SHALL award XP according to:
  Earned XP = Base XP + (Remaining Seconds × 2) + (Remaining Hearts × 50)
- **[DEV-FR-F03-005] Real-Time Global Leaderboard:** The system SHALL maintain a high-performance Redis sorted set computing real-time global ranks, daily ranks, and badge standings across all users.

4.4 Feature 04: Dual-Purpose Developer Community

(Subsystem Lead: Kalhari L.T. — 244096T)

4.4.1 Description

Provides an asynchronous technical blogging engine alongside a low-latency real-time discussion chat platform for peer learning, knowledge dissemination, and mentor recognition.

4.4.2 Functional Requirements

- **[DEV-FR-F04-001] Markdown Technical Article Publishing:** The system SHALL allow users to author, format, preview, and publish technical articles with embedded code blocks, hashtags, and banner images.
- **[DEV-FR-F04-002] Social Engagement & Reactions:** Users SHALL be able to upvote (`BlogLike`), comment in threaded discussions, and follow fellow developers.
- **[DEV-FR-F04-003] Real-Time WebSocket Discussion Rooms:** The system SHALL provide Socket.IO powered chat rooms partitioned by technical topics (e.g., `#react-help`, `#mongodb-architecture`, `#general-lounge`) supporting instant messaging and code snippet exchange.
- **[DEV-FR-F04-004] Mentor Verification & Badging:** The system SHALL display a `VerifiedMentorBadge` on profiles, articles, and chat messages for users whose credentials have been validated by administrators.
- **[DEV-FR-F04-005] Moderation & Safety Reporting:** Users SHALL be able to flag abusive content or spam (`Report`). Posts exceeding 3 unique flags SHALL be hidden automatically pending admin review.

4.5 Feature 05: Hybrid Mentorship Ecosystem & Discount Rewards

(Subsystem Lead: Dinisuru P.W.L. — 244047V)

4.5.1 Description

A two-tier mentorship platform coupling instant AI-powered code assistance with structured 1-on-1 human mentor appointments, managed via a platform coin economy and performance-based reward vouchers.

4.5.2 Functional Requirements

- **[DEV-FR-F05-001] Consumable Coin Wallet:** The system SHALL maintain a `CoinWallet` for every user. Users receive a daily free allocation (e.g., 20 coins) and can purchase coin packages via PayHere.
- **[DEV-FR-F05-002] 24/7 AI Mentor Querying:** Users SHALL be able to query the AI Mentor for hints, code refactoring, and bug diagnosis. Each AI response consumes a fixed number of coins (e.g., 5 coins/query).
- **[DEV-FR-F05-003] 1-on-1 Human Mentor Booking:** Verified mentors SHALL publish available calendar slots (`AvailabilitySlot`). Learners can book slots using platform coins or direct currency checkout.
- **[DEV-FR-F05-004] PayHere Payment Processing:** The system SHALL interface with PayHere to facilitate coin top-ups, verifying hash security signatures and handling webhook payment notifications idempotently.
- **[DEV-FR-F05-005] Performance-Driven Discount Vouchers:** The system SHALL generate dynamic discount vouchers (`DiscountReward`) based on learning milestones (e.g., achieving top 10% leaderboard rank or a 14-day Dojo streak unlocks 20% off mentor booking packages).

5. Non-Functional Requirements (NFRs)

5.1 Performance Requirements

- **[DEV-NFR-PERF-001] AST Parsing Speed:** Client-side AST validation for files under 500 lines of code SHALL execute within ≤ 300 ms.
- **[DEV-NFR-PERF-002] Live Preview Update Latency:** In-browser compilation and preview re-render SHALL complete within ≤ 800 ms after typing ceases.
- **[DEV-NFR-PERF-003] Server Execution Timeout:** Sandbox test runs on the backend SHALL enforce a hard execution cutoff at 3000 ms to prevent memory locks.
- **[DEV-NFR-PERF-004] Real-Time Socket Latency:** Chat messages and simulator game state updates SHALL broadcast to room participants with a latency ≤ 100 ms under standard broadband connections.
- **[DEV-NFR-PERF-005] Leaderboard Refresh Throughput:** Global leaderboard rank calculations utilizing Redis sorted sets SHALL return top-100 queries within ≤ 50 ms.

5.2 Security Requirements

- **[DEV-NFR-SEC-001] Authentication & Password Encryption:** All passwords MUST be hashed using `bcrypt` with a minimum work factor of 12. Authenticated sessions MUST use cryptographically signed JWT tokens with expiration ≤ 24 hours.
- **[DEV-NFR-SEC-002] Sandbox Isolation:** Student code executed in the browser MUST run inside a separate origin or sandboxed `iframe` without access to `window.parent.document` or `localStorage`.
- **[DEV-NFR-SEC-003] Injection & XSS Prevention:** All user-generated markdown, blog posts, and chat messages MUST be sanitized using `DOMPurify` before rendering to prevent Cross-Site Scripting (XSS).
- **[DEV-NFR-SEC-004] Payment Verification Integrity:** PayHere payment notifications MUST verify the MD5 hash checksum: *Checksum Hash = MD5(Merchant ID + Order ID + Amount + Currency + Status Code + MD5(Merchant Secret))*

5.3 Reliability & Availability

- **[DEV-NFR-REL-001] System Uptime:** The platform SHALL maintain ≥ 99.5% operational availability during core academic term evaluation periods.
- **[DEV-NFR-REL-002] Workspace State Resilience:** If a browser crash or network disconnection occurs, the workspace SHALL restore unsaved code from local IndexedDB/localStorage upon reload.

5.4 Usability & Accessibility

- **[DEV-NFR-USA-001] Accessibility Compliance:** User interfaces SHALL conform to WCAG 2.1 Level AA standards, ensuring full keyboard navigability for the Monaco Editor and appropriate contrast ratios for dark theme interfaces.
- **[DEV-NFR-USA-002] Responsive Breakpoints:** All public views, community blogs, and Theory Dojo videos SHALL be fully functional across mobile (375px), tablet (768px), and desktop (1280px+) viewports.

---

6. System Data Models & Database Architecture

DevCraft utilizes MongoDB Atlas for primary document persistence and Redis for volatile high-throughput caching.

6.1 Entity-Relationship Cardinality Overview

Primary Entity	Related Entity	Cardinality	Business Association & Scope
`User`	`UserProjectProgress`	1 to Many (1..*)	Tracks learner progress, completed steps, and time across guided projects.
`User`	`LessonProgress`	1 to Many (1..*)	Records video completions and concept mastery states in the Theory Dojo.
`User`	`DebugSession`	1 to Many (1..*)	Historical log of time-attack debugging attempts, heart losses, and earned XP.
`User`	`Blog`	1 to Many (1..*)	Technical articles authored and published by the user in the community feed.
`User`	`CoinWallet`	1 to 1 (1..1)	Stores user coin balance and transaction ledger for AI & human mentorship.
`User`	`Booking`	1 to Many (1..*)	1-on-1 scheduled appointments between students and verified mentors.
`GuidedProject`	`ProjectStep`	1 to Many (1..*)	Sequential atomic milestones rendered in the Monaco workspace.
`ProjectStep`	`ASTValidationRule`	1 to Many (1..*)	Babel syntax assertions checked in real-time during user editing.
`ProjectStep`	`UnitTestSpec`	1 to Many (1..*)	Behavioral Jest unit assertions executed silently in sandboxed workers.
`Lesson`	`VideoCheckpoint`	1 to Many (1..*)	Timestamped interactive gatekeeper questions embedded in instructional videos.
`DebugChallenge`	`ChallengeTestCase`	1 to Many (1..*)	Hidden unit tests that validate the user's bug fix in the simulator.
`MentorProfile`	`AvailabilitySlot`	1 to Many (1..*)	Weekly time slots published by verified mentors for student booking.

(For full Peter Chen notation models and complete entity attribute lists, see MASTER_ER_SPECIFICATION.md).

6.2 Core Collection Schemas

1. **`users` Collection:** `_id`, `username`, `email`, `passwordHash`, `displayName`, `profileImage`, `totalXP`, `globalRank`, `fatigueScore`, `isVerifiedMentor`, `role`, `createdAt`.
2. **`guided_projects` & `project_steps`:** Project blueprints, step instructions, initial boilerplate, solution code, AST rules, and Jest assertions.
3. **`lessons` & `dojo_puzzles`:** Track names, video URLs, checkpoint timestamps, markdown text, code templates with blank token indexes, and accepted regex answers.
4. **`debug_challenges`:** Buggy code base, prompt description, expected test cases, time limit seconds, base XP bounty, and mock error console logs.

5. **`community_blogs` & `chat_messages`:** Title, slug, markdown body, tags, likes, nested comments, room IDs, and message timestamps.
6. **`coin_wallets` & `mentor_bookings`:** User ID, coin balance, transaction ledger, mentor hourly rates, available calendar slots, PayHere payment status.

---

7. Verification & Acceptance Criteria

7.1 Acceptance Test Matrix

Test ID	Target Feature	Test Scenario	Input / Action	Expected System Behavior
TC-F01-01	Guided Workspace	Correct code submission	Student enters valid Express route matching AST requirements.	AST passes; Jest passes; Step checkmark turns green; Step 2 unlocks; +50 XP awarded.
TC-F01-02	Guided Workspace	Fatigue detection	Student fails test 4 consecutive times within 2 minutes.	System evaluates fatigue score ≥ 0.75; triggers `MemePopupManager`; displays funny debugging meme modal.
TC-F02-01	Theory Dojo	Video checkpoint pause	Video reaches timestamp 02:45 containing a question.	Video halts automatically; fullscreen modal prompts question; playback disabled until correct choice made.
TC-F02-02	Theory Dojo	Code puzzle validation	Student fills blank with `useState(0)` into React hook blank.	Evaluates token against regex; marks input green; updates `ConceptProgress` to mastered.
TC-F03-01	Debug Simulator	Heart penalty & Game Over	Student submits failing code 3 times in time-attack mode.	Hearts decrement 3 → 2 → 1 → 0; session terminates immediately; failure screen displayed; 0 XP awarded.
TC-F03-02	Debug Simulator	Timed victory & XP bonus	Challenge solved with 120s remaining and 2 hearts.	Calculates bounty: Base XP + (120 × 2) + (2 × 50); updates Redis leaderboard; updates user profile.
TC-F04-01	Community	Real-Time Chat message	User posts question in `#react-help`.	Socket server receives event; broadcasts message to all connected clients in ≤ 100 ms; persists to MongoDB.
TC-F05-01	Mentorship	AI Query with Coin deduction	User sends prompt to AI Mentor (Balance: 25 coins).	Gemini streams educational hint; 5 coins deducted; balance updates to 20 coins in wallet UI.
TC-F05-02	Mentorship	PayHere Coin Top-up	User completes LKR 500 top-up via PayHere sandbox.	PayHere sends IPN webhook; backend verifies MD5 hash; credits 100 coins to wallet; transaction logged.

8. Project Timeline & Milestone Phasing (IN2901 Roadmap)

Phase	Milestone Name	Timeline Schedule	Key Deliverables & Validation Criteria
Phase 1	Inception & Architectural Formulation	August – September 2026	Topic selection, feasibility assessment, project proposal submission, and initial architectural draft.
Phase 2	Detailed Design & Specification	September – October 2026	Master Class diagrams, Peter Chen ER data models, and comprehensive IEEE 830 compliant SRS specification.
Phase 3	Subsystem Core Development	Late October – December 2026	Implementation of Workspace IDE (F01), Theory Dojo (F02), Debug Simulator (F03), Community (F04), and Mentorship (F05).
Phase 4	System Integration & Quality Assurance	Late December 2026 – January 2027	End-to-end subsystem integration, security audits, PayHere sandbox tests, and user testing.
Phase 5	Final Academic Evaluation & Demonstration	Late January 2027	Final project demonstration, viva voce evaluation, and project documentation hand-off.

DevCraft · Team 22 (Team DoD)
Faculty of Information Technology · University of Moratuwa
IN2901 — Software Development Project (2026)